Autor: Mailton Olinto de Oliveira Lemos
Github: https://github.com/soturnacrosta/sistemaBancario

Sistema Bancário

O presente documento tem como objetivo levantar as necessidades e o desenvolvimento de uma aplicação Java para Software bancário. Aqui se encontra todo o conhecimento necessário para construir, manter e evoluir o Software.

Requisitos funcionais:

- O sistema deve permitir fazer transações TED entre contas;
- O sistema deve permitir fazer operações de saque e depósito;
- Possuir autenticação com login e senha através de algoritmo ‘hashing’;
- Armazenar dados em um banco de dados;
- Deve permitir operações de extrato;
- Abertura e encerramento de contas através de um gerente (superusuário);
- Alteração de dados bancários por meio de autenticação;
- Visualização dos dados bancários por meio de autenticação;

Requisitos não funcionais:

- Transações devem ser seguras, com mínima tolerância de falha;
- As operações só podem ser realizadas quando logado;
- Operações de saque e depósito não podem permitir valores incompatíveis como valores nulos e negativos;
- Alterações nas contas devem ser estritamente seguras, não permitindo alterações sem autenticação;
- Autenticação robusta, com ‘hashing’ e lógica de conexão SQL em camadas;
- O extrato deve permitir até as últimas dez (10) operações da conta no banco;
- Limite de cinco (05) transações TED diárias.

Requisitos de domínio:

- As informações devem ser bem protegidas, seguindo a LGPD;.
- Visualização de dados e alterações de dados só podem ser acessados quando autenticados (logados) na aplicação;
- Abertura e encerramento de contas só podem ser feitos através de um gerente (superusuário).
- Um banco só pode ter um (1) gerente;
- Devem ser utilizadas formatações padronizadas de mercado como Decimal para moeda e Regex para CPF.

Requisitos de sistema:

- O sistema será escrito em JAVA;
- O banco de dados será escrito em MySQL;
- A autenticação será feita através de algoritmo de ‘hashing’ BCrypt;
- O sistema deve exigir autenticação na tela inicial;
- Tratamento de exceções com ‘Exceptions’ com mensagens amigáveis;

Diagrama de classes:

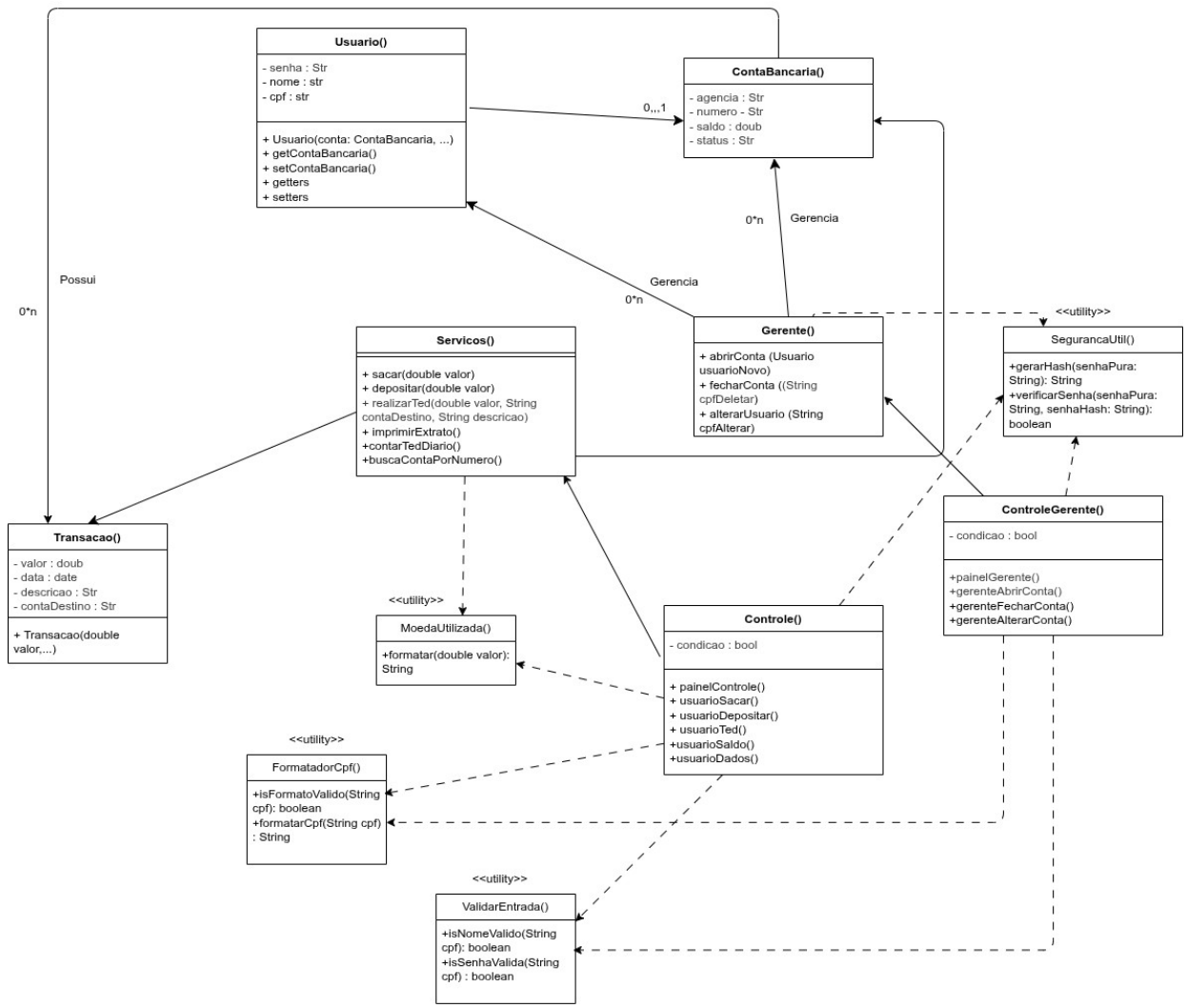

Figura 1: Diagrama de classes do Sistema Bancário

Funcionamento do Software

1. Panorama geral

O software é dividido em doze (13) classes, sendo o cérebro do funcionamento a classe Servicos() onde o usuário poderá realizar as operações bancárias básicas; e o menu interativo ao usuário, a classe Controle() e ControleGerente() que chamam os métodos necessários para executar o sistema inteiramente.

A classes Usuario serve para guardar os dados do cliente. A classe Gerente serve para mexer com um pouco mais de profundidade no sistema (como abrir, alterar ou encerrar contas). O usuário comum não tem acesso a classe Usuario, pois ela apenas guarda os dados dele, o usuário tem contato com o menu em Controle.

O software tem também a classe Transacao onde ocorre as transações TED com registros das informações.

Há duas (2) classes de ‘Exceptions’ sendo uma para entradas inválidas de usuário (EntradaInvalidaException()) e outra para operações com saldo insuficiente (SaldoInsuficienteException()). Isso permite um controle melhor do sistema, padronização de mensagens e uma posterior manutenção mais flexível.

Há uma classe chamada MoedaUtilizada() que serve para formatar os valores ‘double’ em cotação decimal. Há outra classe utilitária chamada FormatadorCpf() que serve para validar e formatar os CPFs inseridos e mostrados.
O projeto está sendo organizado pensando na sua manutenção e evolução e por isso, as classes têm o intuito de delegarem a menor quantidade de responsabilidades possíveis.

1.2 Fluxograma

Aqui está um fluxograma com uma visão geral do sistema, abordando quatro funções básicas como abertura e encerramento de conta, além de transações TED e depósitos.

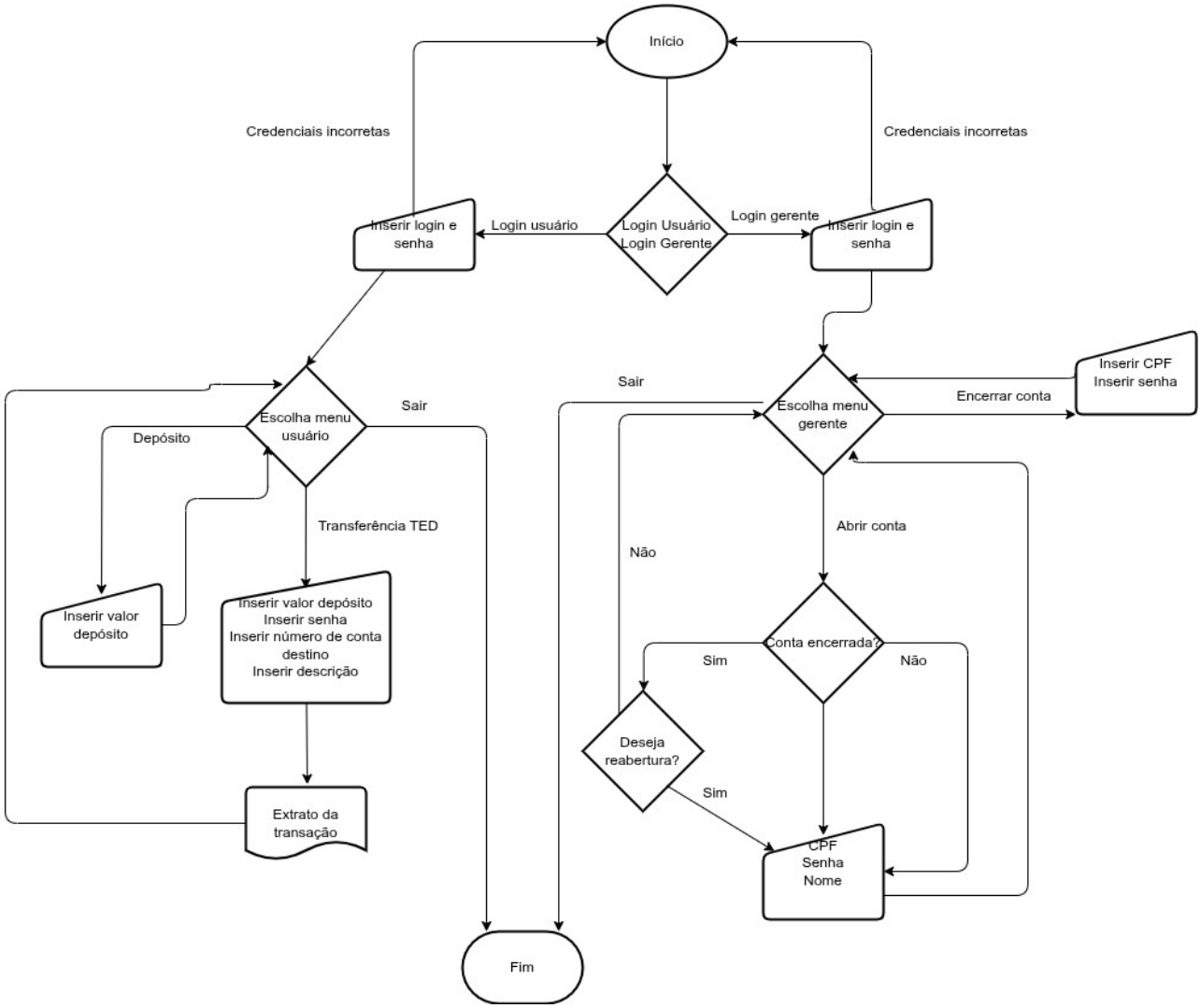

Figura 2: Fluxograma do sistema bancário

2. Classes

2.1 Controle

A classe Controle fica responsável pelo contato com o usuário, permitindo que este opere o software sem nem mesmo precisar entender seu funcionamento por de trás dos panos.
Sua lógica é bem simples:

Implementa uma instância de ContaBancaria() para acessar seus dados.

Implementa uma instância de Usuário() e Gerente() para acessar seus dados e possibilitar login.

Exite um loop através de boolean que permite permanecer no menu até sairmos por vontade própria. O menu principal se desmembra até submenus, através de escolhas numéricas e entradas de dados.

Para efetuar login de usuário, a variável responsável é reiniciada para nula toda vez que entramos no menu de login, fortificando a segurança. São inseridos os dados de CPF e senha, da qual o CPF é formatado para caber no padrão do sistema e em seguida procurado no banco de dados. Após, é comparada a senha digitada e o CPF digitado com os dados correspondentes no banco de dados.

Caso a conta esteja com o status de encerrada, o login é barrado mesmo com credenciais corretas.

Caso as credenciais sejam inválidas, o login é barrado.

Por fim, há funções do tipo void que chamam métodos correspondentes na classe ContaBancaria e Gerente.

É importante destacar o fluxo básico: a classe Controle é primeiramente instanciada, dela são chamadas funções da própria classe, das funções são chamados métodos e instâncias das classes Servicos(), o "cérebro" das operações bancárias, ou ControleGerente() da qual administra as informações de usuário dessas contas.

Os erros de exceções são tratados através das 'Exceptions', sendo as do tipo 'InputMismatchException ' e uma personalizada as duas existentes nessa classe. A primeira serve para evitar entradas inválidas de usuário e a segunda, operações com saldo insuficiente.

É comumente visto a utilização de classes utilitárias que visam formatar entradas e saídas de dados. Exite uma formatação para dados double (a parte monetária) e outra para String (a parte dos CPFs). FormatadorCpf e MoedaUtilizada respectivamente.

2.2 ControleGerente

Aqui é onde o gerente vai interagir com o menu.

Segue a mesma lógica que Controle(), rodando um menu em loop através de uma variável booleana até sua condição ser satisfeita.

Nos submenus, chama métodos da classe Gerente().

2.3 Servicos

A classe Servicos() opera como o "cérebro" do sistema. Nela que são processadas as principais operações do sistema bancário como: sacar, depositar, transações TED e extrato.

Sua lógica é simples:

Implementa uma instância de Transacao() e ContaBancaria() para acessar seus dados.

O método *sacar()* do tipo double recebe parâmetro ContaBancaria contaOrigem, double valor e String senhaSaque. Se o valor passado for maior que o saldo em conta, exibe uma mensagem de erro através de uma 'Exception' e encerra o laço.

Caso o valor de saque for menor que o saldo em conta,. ele executa o cálculo de saque e retorna a função com o print de sucesso.

O método *depositar()* do tipo double recebe parâmetro ContaBancaria contaOrigem e double valor. Se o valor passado for maior que zero (0), ele executa o cálculo de depósito, retorna o a função com print de sucesso.

Caso o valor de depósito for zero (0) ou negativo, exibe uma mensagem de erro e encerra o laço.

O método *realizarTed()* do tipo void recebe parâmetros ContaBancaria contaOrigem, double valor, String contaDestino, String destricao. O valor recebe o valor da transação, contaDestino recebe o número da conta do favorecido e descricao é um espaço para anotar uma descrição e aparecer no extrato.

Ele começa com o contador de transações diárias para obedecer o teto estabelecido, depois um boolean de valor falso com laço for para percorrer a lista de contas bancárias abertas e descobrir se a contaDestino (que foi passada) existe no banco de dados. Caso encontre, o boolean se torna verdadeiro.
Com o boolean verdadeiro, caso o valor for maior que o saldo, exibe mensagem de falha através de uma 'Exception'.
Caso haja sucesso na operação, aplica o cálculo, instancia a classe Transacao e adiciona a lista de transações para registrar no extrato. Exibe uma mensagem de sucesso.

O método *imprimirExtrato()* do tipo void, gera os extratos através do ArrayList de transações registradas.
Ele verifica se a lista está vazia, primeiramente.
Se não estiver vazia, imprime o número da conta, o valor da transação, a data e a descrição inserida pelo usuário através de um laço for que percorre cada transação. Esses dados são obtidos através da classe Transacao.

O método contarTedDiario() dá suporte ao método realizarTed(). Serve de contador para o teto de transações diárias. Ele inicia com um contador de inteiro em zero, importa a biblioteca 'Date' para poder extrair as datas, formata a data extraída para dia, mês e ano e guarda numa String. Faz um faço 'for' percorrendo as transações e busca as datas das transações e compara com a data atual. Retorna o contador para usuarioTed() utilizar.

2.4 ContaBancaria

É uma classe simples, responsável por guardar os dados da conta do usuário. Nela estão guardados o número da conta, que é único e auto-incrementável, o status da conta (ativa ou encerrada), agência, saldo e o CPF do usuário corresponde.
Contas encerradas não são excluídas do banco de dados, apenas recebem o status de "ENCERRADA" e passam a não permitir logins ou receber transferências.

2.5 Transacao

A classe Transacao registra os dados das transações do usuário no sistema e permite a impressão do extrato bancário junto do método *imprimirExtrato()* na classe ContaBancaria.
É uma classe simples, contendo apenas cinco atributos, sendo valor, data, contaDestino, contaOrigem e descricao, correspondendo ao valor da transação, a data da transação, a conta destino, conta origem da transação e uma anotação da transação feita.
Um construtor de classe é inserido para que os dados sejam gravados no extrato.
Quando uma transação é feita, a hora é atual correspondente é extraída para o registro automaticamente com a biblioteca 'java.util.Date'. Assim evita confusões cronológicas que entradas de data manuais poderiam gerar.
Há getters dos atributos.

2.6 Usuario

A classe Usuario fica responsável pelos dados do usuário, tais quais o nome, CPF, senha e sua conta bancária correspondente.
É uma classe de conteúdo simples, contendo apenas sua associação com ContaBancaria, três atributos do tipo String, seus getters e setters.

2.7 Gerente

A classe gerente fica responsável por gerenciar as contas de usuários. Nele você pode abrir e encerrar as contas dos clientes. Sua lógica é simples:

Há três métodos:

abrirConta() que como o nome sugere, abre contas após receber os valores da classe controleGerente(). Ela procura por contas existentes através do CPF digitado, caso exista, verifica se está ativa ou encerrada. Caso não exista um CPF cadastrado, cria a conta, adiciona ao banco de ContaBancaria e Usuario com seus atributos, seta o saldo inicial para zero, seta a agência correspondente e retorna mensagens de validação. Caso já existe uma conta criada, mas encerrada, reativa a conta com novos dados.

fecharConta() que como sugerido o nome, encerra uma conta já existente. Da mesma forma, procura por uma conta no banco de dados através do CPF digitado. Se encontrar uma conta com o CPF, verifica se o saldo está zerado. Se estiver zerado, remove a conta e o usuário do banco de dados retornando uma mensagem. Caso não encontre, retorna uma mensagem de erro.

alterarUsuario() que possibilita alterar o nome do cliente, CPF e senha da sua conta. A sua lógica é simples e envolve procurar o CPF digitado no banco de dados. Se encontrar um usuário válido e com conta ativa, altera o nome, CPF e a senha e retorna mensagem de validação. Caso não encontre um CPF existente no banco de dados, retorna mensagem de erro.

A classe Gerente é alimentada pelos valores que vêm da classe Controle.

3. Exceções

As exceções são padronizadas em classes para melhor controle de fluxo, segurança e comunicação para com o usuário. Vale salientar que isso torna a manutenção do software mais simples.

Existem basicamente duas classes de exceções no sistema: SaldoInsuficienteException e EntradaInvalidaException.

3.1 SaldoInsuficienteException

É uma classe de tratamento de erros. Ela herda a RuntimeException. Primeiramente, ele formata os valores ‘double’ para o formato decimal para padronizar o sistema; em seguida, retorna uma mensagem de erro amigável formatada. Em seguida, ele lança a mensagem formatada para o construtor.

Serve para tratar operações com saldo insuficiente.

3.2 EntradaInvalidaException

Outra classe de tratamento de erros. Herda a RuntimeException. Apenas retorna uma mensagem de erro amigável.

Serve para tratar erros de entrada de usuário. Por exemplo, quando o usuário insere uma String ao invés de um ‘double’.

4. Padronizações

Para padronizar o formato da moeda, optou-se a implementação de uma classe que formate as casas dos valores ‘double’. Nesse caso, escolhe-se a biblioteca ‘DecimalFormat’.

4.1 MoedaUtilizada()

A classe que formata e padroniza as moedas é a MoedaUtilizada. Ela é bem simples e apenas precisa instanciar um método com o formato escolhido e retornar o formato.

4.2 ValidarEntrada()

Serve para aceitar nomes de usuários que sejam apenas letras, sem dígitos.

Apenas retorna o nome formatado.

4.3 FormatadorCpf()

Serve para formatar os CPFs inseridos para o padrão convencional e também para barrar entradas inválidas de CPF.
Apenas precisa retornar a formatação de caracteres e dígitos.

5. Autenticações

Na primeira versão do software, as autenticações foram feitas através do percorrimento de lista de Usuario em laços ‘for’, comparando a senha inserida pelo usuário com a senha guardada na lista ‘Static’ através do método get(), no caso dos usuários.
No caso da autenticação do gerente, se compara a entrada de dados com uma String para senha e outra para CPF já preestabelecida.
Os dados de acesso do gerente são:

Nome: admin
Senha: admin
CPF: admin

Também foi utilizados lógicas ‘If’ para comparar ‘getSenha()’ com e ‘equals’ da senha do usuário autenticado com a entrada de dados do usuário naquela operação. Ao realizar saque, por exemplo, digita a senha e ela é passada para comparar com a senha do próprio usuário autenticado, diretamente.

6. Número de agência

A agência é definida no código e é fixa, sendo impossível de ser inserida pelo gerente ou usuário na hora da abertura de uma conta.
Para modificar o número de agência para utilizar o sistema em agências diferentes, basta entrar na classe Gerente() e na linha que consta ‘contaNova.setAgencia()’ trocar o parâmetro String pelo número de agência desejado.
Isso impede erros de digitação na abertura de conta, delegando a responsabilidade para equipes especialistas na implementação do sistema.

7. Consistência dos dados em memória

Enquanto não utilizado um banco de dados real, a inicialização dos objetos deve ocorrer antes da chamada dos métodos de busca para garantir que o laço ‘for’ encontre os registros.
Para garantir a persistência dos dados durante a execução, optou-se pelo uso de membros estáticos nas coleções de usuários e contas, simulando um repositório centralizado de informações.

8. Gerenciamento de fluxo

O sistema implementa o esvaziamento do buffer do Scanner (‘input.nextLine()’) imediatamente após capturas de tipos primitivos (double, int), prevenindo a reexecução involuntária do laço ‘while’ e garantindo a estabilidade dos submenus de saque e depósito.

9. Testes unitários

Para garantir a robustez e segurança do sistema, foram aplicados testes unitários em pontos estratégicos através do JUnit 5.
As classes Gerente, ContaBancaria e Controle guardam lógicas cruciais que precisam de cuidado em dobro e por isso, a validade dos testes foram confirmadas.
Os testes ficam na pasta ‘sistema-bancario\src\test…’ e as classes testas consistem em:

GerenteTest, na qual o método validado foi abrirConta(). Confirmou-se que não se aceita abrir duas ou mais contas para um mesmo CPF.

Servicos, na quais os métodos validados foram aceitaDepositoNegativo(), aceitaSaqueNegativo() e aceitaTedInvalido(). Todos os três testam se o sistema aceita dígitos inválidos na hora da sua devida operação. Há um quarto método chamado aceitaTedNull() da qual verifica se é possível realizar uma transferência para uma conta não cadastrada. Todas as verificações foram passadas no teste.

Em ControleTest, validaram-se estaLogado() e testeLoginFalho(), da qual é verificado se a variável de login é reiniciada sempre que entramos na tela de login, ou seja, quando estamos deslogado. E a segunda, impede o login com a senha incorreta.
Sempre que for implementadas lógicas de fluxo, segurança e negócio cruciais, vê-se necessário a implementação estratégica desses testes. Basta replicar os testes unitários com JUnit 5.

10. Guia de manutenção/evolução

Todas as dependências necessárias para evolução do sistema estão no arquivo pom.xml na raiz do projeto.

10.1 Padrões de projetos

O projeto foi pensado para ser fácil de manter e evoluir. Para isso, dois padrões de projetos principais foram implementados:

- *Singleton*
- *Guard Clause*

Em outras palavras, as classes foram pensadas para delegarem o menor número de responsabilidades possíveis, enquanto os métodos são mais limpos e fáceis de entender ao evitarmos a construção de lógicas de ‘if-else’ aninhados.
Ao necessitar de implementar novas funcionalidades, busque delegar a responsabilidade à classe adequada sem sobrecarregá-la com excesso de responsabilidades. Caso necessário, crie mais classes.
Ao necessitar de implementar camadas de segurança e lógica, busque fazer um código mais limpo e “vertical”, através das cláusulas de guarda. Até no máximo três ‘if-else’ são válidos implementar.

10.2 Migrando para o MySQL

As listas ‘Static’ que até então serviam para armazenar os valores das variáveis, foram substituídas por uma classe Conexao() e ‘queries’ SQL. O sistema foi inteiramente refatorado para suportar buscas, criações, atualizações e exclusões SQL.
Na ideia inicial do sistema, suportada por listas, ContaBancaria() guardava atributos, continua listas e operava lógicas de negócio. Isso gerava um excesso de responsabilidades que o projeto objetiva evitar.
Na migração para o banco de dados, foram excluídas as classes originais e criadas as suas versões ‘DAO’ e ‘BEAN’. A classe ContaBancaria() deixou de delegar suas três funções ao ser criada uma classe chamada Servicos(), onde as lógicas de negócio são aplicadas.

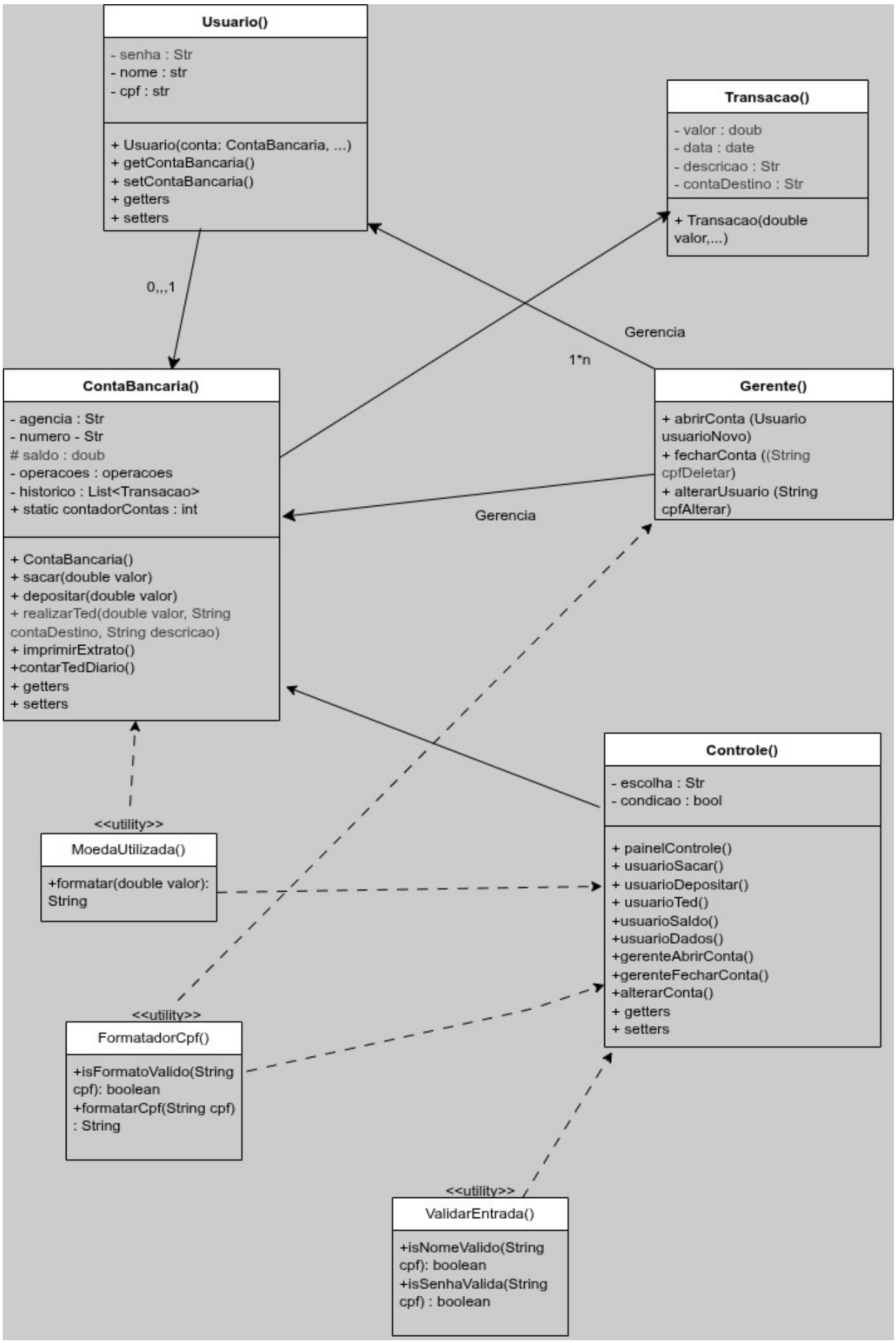

Figura 3: Diagrama de classes inicial (antigo)

11. Banco de dados

Para armazenar os dados dos clientes e administradores em um lugar seguro, temos os bancos de dados. O ‘MySQL’ foi escolhido pois reflete melhor o mercado real de serviços bancários, evitando gargalos em operações simultâneas em mesma conta.
O diagrama foi modelado pelo MySQL Workbench e a construção, testes e gerenciamento das tabelas físicas foram feitos pelo mesmo.

11. 1 Diagrama lógico

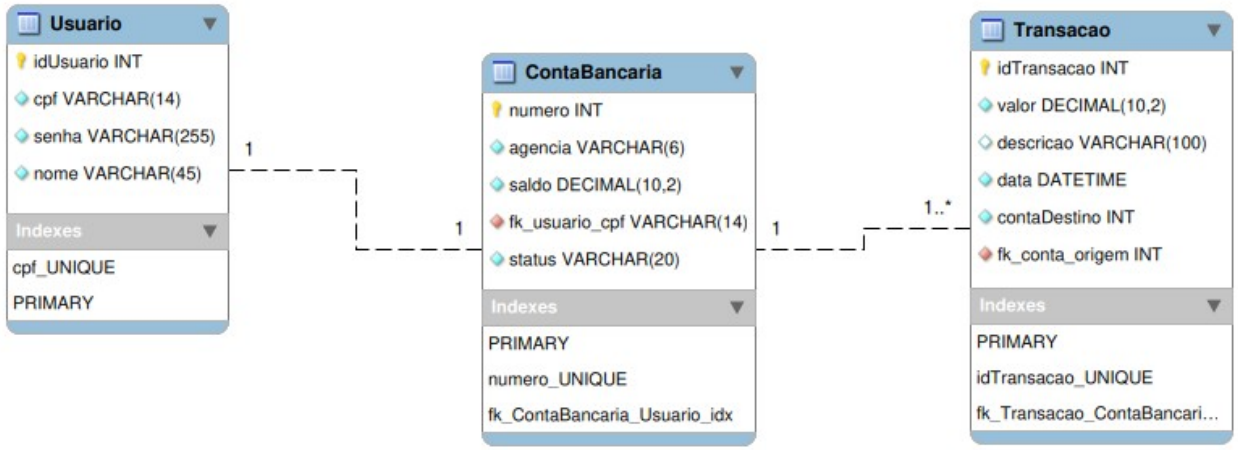

Figura 4: Diagrama lógico do banco de dados do sistema bancário

11. 2 Lógica por trás do banco de dados

A tabela ContaBancaria segue a seguinte lógica:

- Uma conta bancária pode ser utilizada por apenas um usuário.
- Essa conta possui um número de conta do tipo inteiro que incrementa a cada criação. Essa é a chave primária.
- A conta possui uma agência designada na implementação do sistema.
- Possui saldo do tipo decimal, que lidar melhor com lógicas financeiras.
- A tabela possui uma chave estrangeira para se interligar com a tabela Usuario e outra para se interligar com a tabela Transacao.
- Possui status da conta, que verifica se está encerrada ou ativa.

A tabela Usuario segue a seguinte lógica:

- Um usuário pode ter apenas uma conta aberta.
- O usuário precisa ter seu CPF cadastrado, que serve para identificá-lo na ContaBancaria.
- O usuário possui uma senha de tamanho 8 no sistema, mas que no banco de dados utiliza VARCHAR(255) para garantir compatibilidade com algoritmos de hashing modernos como BCrypt.
- O usuário possui nome.

A tabela Transacao segue a seguinte lógica:

- Uma transação pertence a uma única conta.

- A tabela tem uma chave primária chamada ‘idTransacao’ do tipo inteiro que incrementa a cada nova transação.
- Possui o atributo ‘valor’, do tipo decimal, que lidar melhor com lógicas financeiras.
- O atributo ‘descricao’, que não é obrigatório, que serve para o usuário inserir descrições nas transações.
- O atributo ‘data’, que registra a hora exata da transação.
- contaDestino que armazena a conta que recebe os valores da transação.
- contaOrigem que armazena a conta remetente da transação. Essa é a chave estrangeira que interliga ContaBancaria e Transacao.

11.3 Tabelas físicas

O código das tabelas físicas do banco de dados está disponível no Github do projeto. A construção do banco de dados é simples e basta copiar e colar o código já escrito.
É importante pontuar que na tabela de ContaBancária há uma linha adicionada ao fim que possibilita iniciar os números de conta a partir de mil (1000):
```AUTO_INCREMENT = 1000;```
OBS: Note que em versões Linux o MySQL Workbench pode dar erro 1064, de sintaxe. Basta apagar as palavras ‘VISIBLE’ do código.

12. Hashing e segurança

O sistema foi criptografado através de ‘hashing’ Bcrypt. O Bcrypt permite gerar novos códigos hashing constantemente, além de ser propositalmente lento. Isso evita possíveis ataques maliciosos e assegura a integridade dos dados dos clientes.
A sua implementação é simples, bastando adicionar as dependências, criar uma classe utilitária e adicionar aos códigos que comparam senhas.
O Bcrypt é suportado pela classe utilitária SegurancaUtil().
```